Design and Evaluation of a Massively Parallel Matrix Multiplication Algorithm Using MPI and CUDA

Kaden Kwan, Ryan Lima, and Ryan Koester
Rensselaer Polytechnic Institute
{kwank3, limar2, koestr}@rpi.edu

ABSTRACT

Matrix multiplication is a computationally intensive operation, with a complexity of $O(n^3)$, making efficient parallelization essential for any large scale application. In this work, we present a hybrid parallel matrix multiplication algorithm that combines MPI-based distributed computing with CUDA-based GPU acceleration. Our approach distributes matrix rows across MPI processes while utilizing GPU kernels with optional shared memory tiling and matrix transposition to optimize computation.

We evaluate performance using both strong scaling and weak scaling experiments, run with 2 to 32 MPI ranks. Our results show that shared memory optimization provides the most consistent performance improvements, while transposition introduces additional overhead but can improve computational intensity. The strong scaling experiments show limited gains beyond a small number of ranks due to communication overhead. Weak scaling reveals increasing runtimes that are dominated by MPI costs. Overall, our findings highlight that while GPU acceleration helps significantly improve computation time, system performance is ultimately constrained by communication overhead and memory access efficiency.

1 INTRODUCTION

Matrix multiplication is a fundamental operation in many computational domains. This includes data analysis, scientific computing and also machine learning. As the matrix size grows, the computational cost of matrix multiplication increases cubically. This makes efficient parallelization essential for any practical application. Modern high performance computing systems provide two sources of parallelism: distributed memory parallelism across nodes and the GPUs within each node.

With our project, we sought to design and implement an efficient hybrid CUDA and MPI parallel matrix multiplication algorithm. Our solution utilizes MPI for inter-process communication and CUDA to accelerate the computation. The goal is to evaluate the performance gains achieved by leveraging distributed computing with GPU acceleration to improve performance and scalability with larger matrices.

In particular, our implementation focuses on splitting up the computation across MPI ranks by distributing rows of the left matrix, while using GPU kernels to accelerate the local computational work. We also chose to investigate how certain optimization techniques affect performance, such as shared memory usage and matrix transposition. This is in an effort to improve memory access efficiency and speed up the runtimes. Performance is evaluated by measuring the total runtime, as well as specifically GPU computation time under varying configurations.

2 BACKGROUND

The standard sequential algorithm for multiplying matrices computes the product of two n x n matrices in three nested loops to achieve a time complexity of $O(n^3)$. This method can be relatively simple to implement, but is inefficient and prohibitively expensive as matrix size grows. In practical simulations, performance is also further limited by memory behavior, in particular memory bandwidth and cache utilization [1].

A key challenge in optimizing matrix multiplication comes from the interplay between computation and memory management. Modern processors rely heavily on cache hierarchies, and inefficient memory patterns can significantly degrade performance. In particular, accessing non-contiguous memory leads to frequent cache misses and increased latency. Some naive implementations that iterate over matrix columns can often suffer from poor cache locality, which leads to excessive memory transfers and reduced throughput [1].

2.2 MPI: CPU-Based Parallelism

Parallelization is widely used to help mitigate the computational cost of matrix multiplication. On distributed memory systems, the Message Passing Interface, or MPI, is a common model. MPI distributes data across processes, each with its own

local memory. It requires explicit communication between nodes, and typically matrices are partitioned into blocks or rows. Each process then computes a portion of the result before exchanging intermediate data. The performance of these systems can depend heavily on communication patterns and data distribution.

2.3 CUDA: GPU-Based Parallelism

GPUs provide another powerful tool for matrix multiplication. By using frameworks such as CUDA, computations can be executed across thousands of lightweight threads. Compared to CPUs, GPUs are designed for this kind of throughput and can truly excel in massive data parallelism in their workloads.

Across both CPU and GPU implementations, a recurring theme is that naive parallelization is insufficient for high performance computing. It is not enough to simply divide work among different processes to improve efficiency. The speedup gained due to such implementations is limited, while more memory forward methods, such as RR and MT discussed by Tomikj and Gusev, significantly improve performance by minimizing memory transfers [1].

3 DESIGN OVERVIEW

The overall design follows a hybrid parallel model which combines both MPI-based distribution and CUDA-based computation. On a high level, the program does the following:

1. **MPI Initialization**
   Each MPI process initializes and determines its rank and total number of processes. Each rank dynamically selects a GPU using cudaGetDeviceCount() and assigns itself a device using rank % deviceCount, which helps ensure portability across various systems.
2. **Data Distribution**
   Rank 0 generates the full input matrices, and the left matrix is partitioned row-wise. It is distributed across processes evenly while the right matrix is broadcast to all processes as well.
3. **Local Computation on GPU**
   Each process performs matrix multiplication on its own assigned rows using a CUDA kernel. The kernel implementation supports optional additional optimizations such as shared memory tiling and transposition of the right-hand matrix.
4. **Result Collection**
   The partial results from each process are sent back to the root process, rank 0, which assembles the final output matrix.
5. **Timing and Evaluation**
   The program measures both GPU kernel execution time as well as total runtime, which includes communication, computation and synchronization.

Our design enables scalable execution across multiple nodes, while leveraging GPU acceleration within each node.

4 ALGORITHM DESIGN

The algorithm itself is based on a distributed row-partitioned matrix multiplication strategy.

4.1 Tiling Strategy

The left matrix A is partitioned evenly across MPI processes by row. Each process computes $C_i = A_i \times B$ where A_i is a subset of rows of A. The number of rows assigned to each process is
(left rows / size), and the final process handles any remainder. This ensures proper load balancing across processes. In addition to this, we use a standard matrix tiling strategy where the distributed matrices are further divided into evenly-sized tiles for that process, which allows different sections to be multiplied in parallel. We chose to use tiles that are 32 by 32 elements, as they have a large area while still being manageable for parallel processing.

4.2 Communication Strategy

For communication, two primary steps are used here: broadcasting of matrix B and the scatter of matrix A. The right hand matrix B is broadcast row-wise to all processes using MPI_Bcast(). If transposition is enabled, the broadcast uses the transposed layout. The root process, rank 0, distributes rows of A using MPI_Send() for non-root ranks, and a local copy for rank 0.

The results are then gathered as each process sends its computed rows back to rank 0. This is done using MPI_Send from the worker ranks and MPI_Recv on rank 0.

4.3 Parallel Computation Model

Each MPI rank uses parallelMultiplication(), which performs matrix multiplication on the GPU for the locally assigned rows. The algorithm supports standard multiplication with global memory access, shared memory optimization of tiling, and optional transposing of matrix B.

4.4 Transpose Optimization

When enabled, the right hand matrix is transposed on the GPU before the computation using transposeMatrix(). This changes memory access patterns so that row-wise access will replace column

accesses. The hope is to improve memory handling and coalescence. However, this also has the potential to introduce additional overhead related to allocation, transformation and communication. We attempt to mitigate this overhead by parallelizing the process. By using tiling with multiple threads, cache misses are reduced and elements are swapped in unison for greater efficiency [2].

4.5 Synchronization and Timing

Synchronization is done using MPI_Barrier(MPI_COMM_WORLD) to ensure consistent data and correctness across multiple processes. Timing is measured as both parallel runtime and total runtime. Parallel timing includes GPU computation time only, while we include communication, computation and synchronization in our total runtime. The total runtime here is measured from before the broadcast and scatter to after the result gathering.

5 IMPLEMENTATION

The implementation combines MPI-based distributed computing with CUDA-based computation in an integrated workflow to achieve efficient large scale matrix multiplication. The program is structured so that MPI is responsible for process coordination, data distribution and result collection. The CUDA portion of the program is used to accelerate the computationally intensive matrix multiplication on each node. Our design utilizes modularity and focuses on a clear separation between communication and computation logic.

5.1 MPI Implementation

Mpi.c handles communication, timing and overall orchestration of the program. GPU devices are assigned by having each MPI process select a GPU dynamically. This is done with the following:

```
cudaGetDeviceCount(&deviceCount);
device = rank % deviceCount;
cudaSetDevice(device);
```

This is done in an effort to improve portability of the program and balance GPU utilization.

5.2 CUDA Implementation

Our CUDA-based computation is encapsulated in the function parallelMultiplication(). Each MPI rank invokes this function to compute the matrix multiplication for their own assigned rows. The kernel execution model follows $C_i = A_i \times B$, operating on the local portion of the matrix. This allows each process to perform computation independently once data distribution is complete. For each process, the problem is broken up by assigning one thread block to each tile.

For parallel transposing, the blocks are assigned to B's tiles, with one thread per row of that tile. Each thread maps the elements in their unique row to the transposed position in the output transpose matrix. Creating a new matrix to store the transpose adds memory overhead, but is required for this implementation to properly handle rectangular matrices.

For parallel matrix multiplication, the thread blocks are assigned to the tiles of the output matrix. Each block has $(\text{tile width})^2$ threads, with each one being responsible for a single element in the tile. This means that each thread has a unique row and column. For non-shared memory, the thread just computes the dot product for its row in A and column in B. For shared memory, each thread loads the values associated with their position from tiles in A and B into shared memory. The dot product using values in those tiles is computed, and this operation is repeated until each tile in A and B used for that output tile have been loaded and multiplied.

Since each tile is handled by a single block, and our grid size is 1024 blocks, grid striding is used to ensure that large matrices are fully covered. We did this by increasing each block's id once it was finished and redoing the process. The new id maps the threads to new rows and columns that have not been previously computed. Grid striding repeats until a block is fully out of bounds, at which point it exits. This is used for both transposing and matrix multiplication.

Parallel Transpose pseudocode:

transposeMatrix(rows, cols, original, result)

Parameters:

rows = The number of rows in the original matrix
cols = The number of columns in the original matrix
original = the matrix being transposed
result = the matrix storing the transpose, already allocated

Constants and keywords:

TILE_WIDTH = The width of each square tile
threadIdx.x = The threads' x index within the thread block
blockIdx.x = The block's x index within the grid
gridDim.x = The number of blocks in the grid

Algorithm:

```
1. blockId = blockIdx.x
2. while(block in bounds):
3.     row = ((blockId / blocksPerRow) *
```

```
            TILE_WIDTH) + threadIdx.x
    col = (blockId % blocksPerRow) *
            TILE_WIDTH
    for i from 0 to TILE_WIDTH - 1:
      result[col+i][row]= original[row][col+i]
    end for
    blockId += gridDim.x
  end while
```

Parallel Matrix Multiplication pseudocode:
parallelMultiplication(rows, shared, cols, left, right, result, useSharedMem)

Parameters:
rows = the number of rows in the left matrix
shared = the shared dimension of left and right matrices (num columns of left and num rows of right)
cols = the number of columns in the right matrix
left = the left matrix
right = the right matrix
result = the product matrix
useSharedMem = if true, use shared memory

Constants and keywords:
Same as above

Algorithm:

```
blockId = blockIdx.x
while(block in bounds):
    firstRow = (blockId / blocksPerRow) *
                TILE_WIDTH
    firstCol = (blockId % blocksPerRow) *
                TILE_WIDTH
    rowOffset = threadIdx.x / TILE_WIDTH
    colOffset = threadIdx.x % TILE_WIDTH
    row = firstRow + rowOffset
    col = firstCol + colOffset
    sum = 0
    if(useSharedMem):
        __shared__ leftShared[TILE_WIDTH][TILE_WIDTH]
        __shared__ rightShared[TILE_WIDTH][TILE_WIDTH]
        for each tile along shared dimension:
            load left tile into leftShared
            load right tile into rightShared
            for i from 0 to TILE_WIDTH - 1:
                sum += leftShared[rowOffset][i] *
                        rightShared[i][colOffset]
            end for
        end for
    end if
    else:
        for i from 0 to shared:
            sum += left[row][i] * right[i][col]
        end for
    end else
    blockId += gridDim.x
end while
```

5.3 Hybrid Integration

The integration between MPI and CUDA is handled explicitly with the main program flow, rather than through separate modules. Each MPI process is responsible for both communication and computation, and a key part of the design is the dynamic assignment of GPUs to MPI ranks, mentioned above.

5.4 Memory Management

Memory allocation is handled through our helper function allocateMatrix(). This function is used to consistently allocate matrices across the program. These include local portions of the left matrix, (leftLocal), the full right matrix (right), local result buffers (resultLocal), and the final gathered result (finalResult).

Allocation is performed per MPI process, meaning each rank independently allocates its needed memory. This allows for memory usage that scales with the number of processes and avoids unnecessary duplication of data structures. Error handling is built directly into allocation calls as a safeguard against exceeding available memory.

Deallocation is done with freeMatrix(), and only the memory owned by each process is freed. Only rank 0 frees the full matrices, while all other ranks free their local buffers. Additionally, when transposition is enabled we free the original right matrix and replace it with the transposed version:

```
freeMatrix(shared,(void**)right);
right = rightTranspose;
```

The implementation also supports an optional shared memory mode (useSharedMem). When enabled, the kernel uses tiling to load blocks of input data into shared memory, reduce global overhead, and improve memory bandwidth. This optimization is particularly useful for large matrices since it reduces the cost of repeated memory accesses.

6 EXPERIMENTAL SETUP

These experiments were performed on the AiMOS system's DCS Cluster. More specifically, we ran on the el8-rpi partition. The compute nodes there each contain two IBM Power 9 processors with 3.15 GHz clock speed and 20 cores. In addition, there is 512 GiB of RAM and 4 NVIDIA Tesla V100 GPUs with 16 GiB of memory each [3]. We allocated four nodes for each test, so the first 16 ranks had their own GPU, while 32 ranks resulted in 2 ranks per device. We used MPI version 16.01 and CUDA version 11.2 for the compilation of our program, which was written in C.

The matrices being multiplied were type shorts, and the product matrix was ints, in the interest of conserving space. For weak scaling, the size of the left matrix was altered while the right matrix was not to ensure the work per rank was consistent. For strong scaling, we used square matrices of 16384 by 16384. The largest matrices we could allocate were 49500 by 49500, though we chose a smaller size due to the high traffic on the AiMOS system.

We primarily measured execution time, both specific to the kernels and the overall runtime, to show trends in performance. We analyzed this data to determine speedups and efficiency as a result of various parameters.

7 EVALUATION

In this section, we provide the resulting charts from our strong scale studies and weak scale studies. For our strong scale studies, we will be keeping the matrix size fixed while increasing the number of MPI ranks. For our weak scaling studies, we will be doubling the amount of rows in one matrix while increasing the number of MPI ranks.

7.1 Strong Scalability

For our strong scaling experiments, we keep our parameters fixed at 16384 for the number of leftRows, shared, and rightCols parameters while doubling the number of MPI ranks from 2 to 32 and increasing the number of GPUs accordingly. We do this for four different cases: with no shared memory and no transpose, with only shared memory, with only transpose, and with both shared memory and transpose.

Figure 1 shows execution time for strong scaling without shared memory or transposition. As MPI ranks increase from 2 to 32, the parallel (GPU) runtime initially decreases significantly, which indicates effective distribution of computation across GPUs. However, beyond a small number of ranks, between 4 and 8, the total execution time begins to increase. As shown in Figure 1, total execution time increases beyond 8 MPI ranks despite continued decreases in parallel runtime.

Compared to the baseline in Figure 1, Figure 2 demonstrates that shared memory reduces both parallel and total execution time across all MPI ranks. This divergence between parallel time and total time highlights the growing impact of MPI communication overhead. While computation becomes faster due to smaller workloads per GPU, the cost of distributing data and gathering results outweighs these gains at higher ranks. The graph demonstrates that naive parallelization without memory optimization leads to diminishing returns.

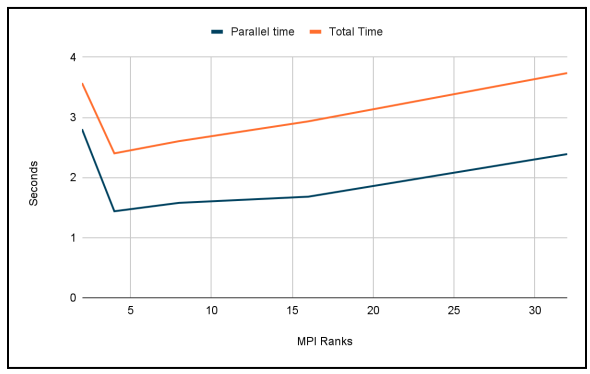

Figure 2. Execution time with only shared memory

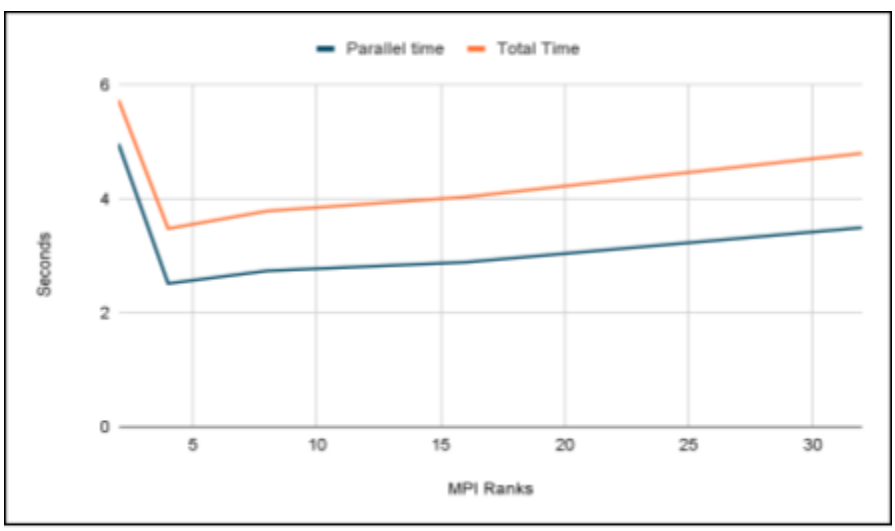

Figure 1. Execution time without shared memory and without transpose

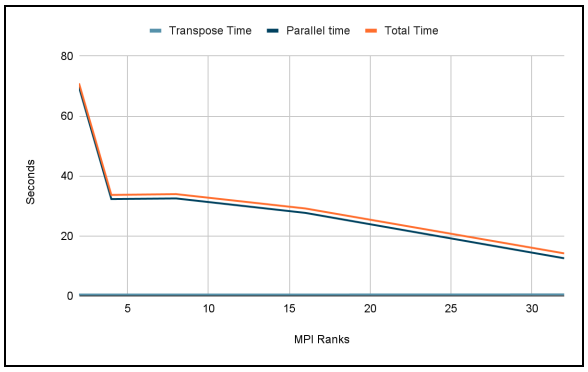

Figure 3. Execution time with only transpose

Figure 3 shows that while transposition reduces parallel runtime slightly, the overall execution time remains higher than the shared memory configuration due to added data movement overhead.

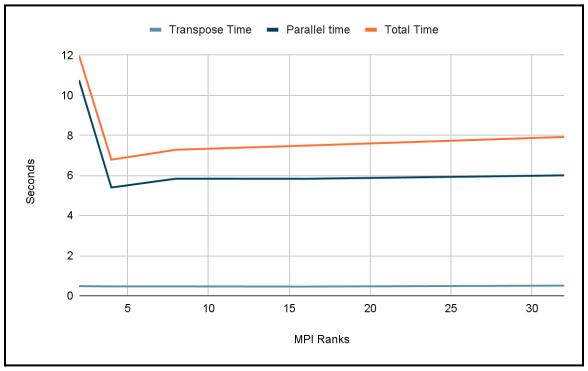

Figure 4. Execution time with shared memory and with transpose

7.2 Weak Scalability

For our weak scaling experiments, we double the rows of the left matrix starting from 65536 to 1048576 while keeping the other parameters as 1024 and doubling our MPI ranks from 2 to 32. We do this for four different cases: with no shared memory and no transpose, with only shared memory, with only transpose, and with both shared memory and transpose.

Figure 5 presents weak scaling results without shared memory or transposition. Ideally, weak scaling should maintain constant execution time as both problem size and number of ranks increases proportionally. However, the figure shows a nearly linear increase in total runtime as MPI ranks increase.

In particular, the parallel runtime remains relatively small compared to total time, indicating that computation is not the bottleneck. Parallel time remaining nearly flat as total time increases linearly indicates that computation scales well, but communication costs grow linearly and dominate execution time. As the number of ranks grows, the cost of broadcasting matrix data and gathering results increases significantly, producing poor weak scaling performance. This confirms that without optimization, distributed overhead compromises execution time even when work per process remains constant.

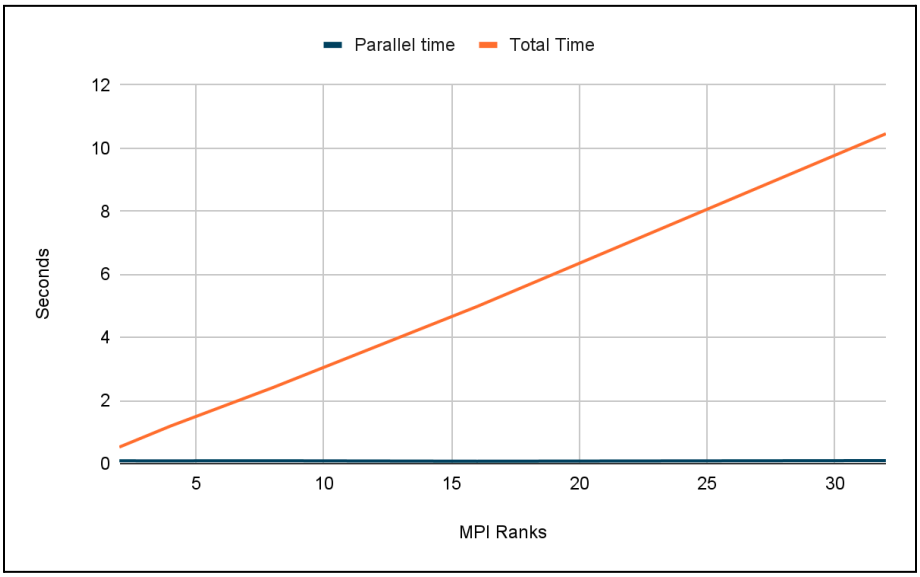

Figure 5. Execution time without shared memory and without transpose

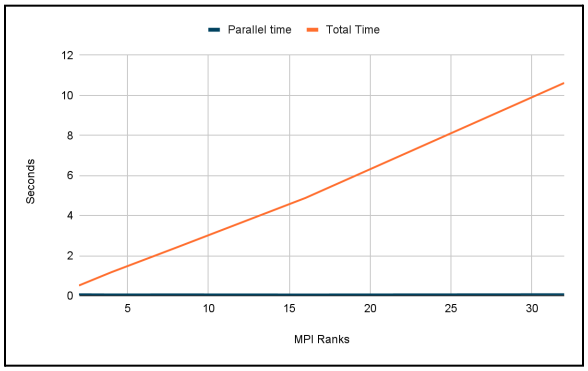

Figure 6. Execution time with only shared memory

Figure 6 shows that shared memory provides only marginal improvements in weak scaling scenarios, as total runtime continues to grow with MPI ranks due to communication costs.

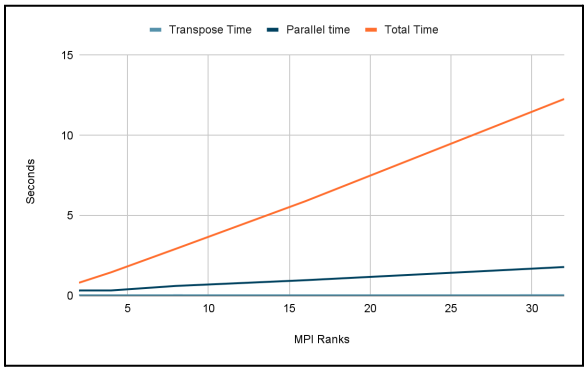

Figure 7. Execution time with only transpose

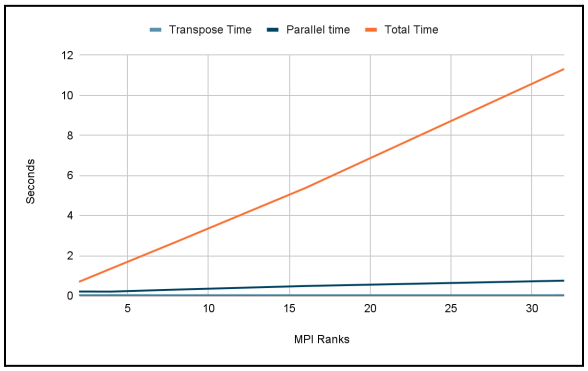

Figure 8. Execution time with shared memory and with transpose

Figure 8 reinforces that even with both shared memory and transposition enabled, weak scaling performance remains limited due to communication overhead.

7.3 Caching

The caching tests are a variation of the original tests designed to solely measure the effect on performance that different indexing strategies result in. Our assumption is that the indexing strategy is directly tied to cache efficiency. No actual transposing was performed for these tests. The execution times reflect only the matrix multiplication for that setup. The parallel runs were done without MPI and using global memory on a single GPU. The serial runs were done on a single CPU, using the typical triple nested for loop. The parallel runs used square matrices up to 16384 by 16384, while the serial runs only went up to 4096 by 4096. There were four unique setups performed for both parallel and serial tests: Normal, right transposed, left transposed, and both transposed. Normal refers to the usual indexing used to calculate the dot product, and transposing swaps the indices accordingly. These setups cover all possible ways to traverse the rows and columns, giving insight to the best method. The indexing is shown explicitly below:

<u>Normal</u> - Left[row][k] * Right[k][col]
<u>Right transposed</u> - Left[row][k] * Right[col][k]
<u>Left transposed</u> - Left[k][row] * Right[k][col]
<u>Both transposed</u> - Left[k][row] * Right[col][k]

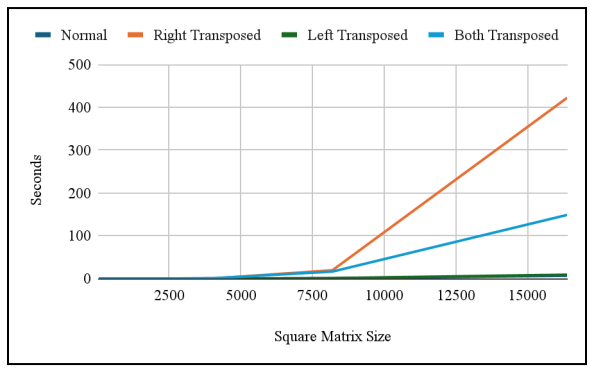

Figure 9. Execution time for parallel without shared memory

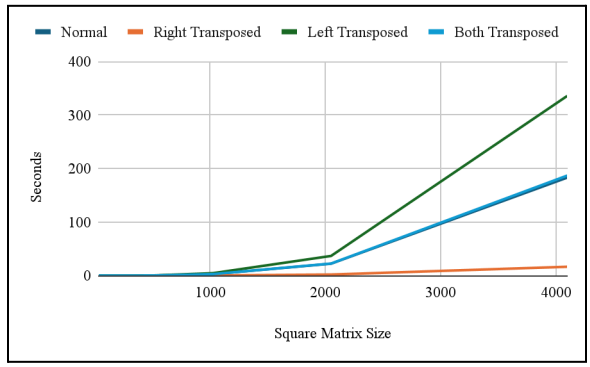

Figure 10. Execution time for serial

7.4 Parallel Overhead Analysis

The performance of the hybrid MPI + CUDA implementation is strongly influenced by two sources of overhead: MPI communication and GPU memory operations.

First, MPI communication overhead plays an impactful role in performance, particularly in weak scaling scenarios. As shown in figure 5, total runtime increases linearly with the number of ranks despite constant workload per process. This overhead comes from broadcasting matrix B, distributing rows of matrix A and finally gathering results.

Second, GPU memory transfer and access costs increase overhead. Although GPU computation is efficient, performance is limited by data transfer costs. Shared memory reduces this overhead, while transposition introduces additional data movement. Together, these two factors show that the system is communication bound and not compute bound, especially at higher levels of parallelism.

7.5 Performance Comparison

Comparing different configurations shows several noteworthy trends in performance. First, GPU accelerated parallel execution significantly outperforms sequential CPU based matrix multiplication, particularly for large matrices. The use of CUDA enables massive parallelism, which reduces computation time significantly. Across our tested configurations, shared memory optimization consistently provides the best performance improvements. By reducing global memory accesses and improving data locality, faster computation across all scaling scenarios is achieved. Conversely, we found that transpose based optimization shows mixed results. While it can improve memory access patterns in theory, it introduces additional overhead that can degrade performance in smaller workloads. Finally, the hybrid MPI + CUDA approach demonstrates that combining distributed and GPU parallelism provides scalability, but only up to a point. Beyond a small number of ranks, communication overhead limits any further gains, which makes pure scaling impossible without introducing additional optimizations.

8 DISCUSSION

Our program scaled the best when using shared memory without transposing, and when each GPU was given reasonably sized matrices to work with. First, it's important to analyze the performance for weak scaling. The overall runtime was linearly proportional to the number of ranks. Weak scaling ensures that each rank does the same amount of work for each test. So, this increase in execution time must be caused by communication overhead from MPI. This is backed up by figure 5, where going from 2 to 32 ranks resulted in a 0.007 second increase in parallel computation time, while total time increased by 9.934 seconds. This means that MPI was responsible for 99.92% of the slowdown in this case. Other figures reveal similar trends, though transposing closes the gap a bit.

While MPI led to poor weak scaling, strong scaling did occasionally have positive results. Namely, it was better when going from 2 to 4 ranks, or when transposing was involved. The change from using 2 versus 4 ranks is the result of optimal load balancing between the ranks. Each rank uses its own GPU, with each one capable of doing work at a certain speed. With 2 ranks, the matrices are too large to be done efficiently, whereas 4 ranks can compute the product quickly.

However, increasing the ranks further is often detrimental. The matrices become too small and the problem is trivial. The GPU stays below peak performance when multiplying in this case. Additionally, with more ranks involved located on different compute nodes, the communication overhead from before is increased. The multiplication speeds the program up less than MPI slows the program down, resulting in a net performance decrease. Transposing does not exhibit this behavior because, as figure 9 reveals, it is very slow. This means that small matrices are still intense computation-wise, allowing more GPUs to give meaningful benefits. In the end, our algorithm is bottlenecked by communication overhead between ranks and limited memory bandwidth, since the majority of work done involves fetching data from global or shared memory.

The caching tests also give valuable data regarding the efficiency of the algorithm. The serial tests confirm what we know should be true: using the right's transpose is significantly faster. This is because C is a row major language, so memory in a row is stored sequentially. Sequential data cached correctly much more consistently. When iterating through columns, as required when using the original right matrix, many cache misses occur where the data has to repeatedly be pulled from RAM due to the large jumps in index. However, it's clear from all transpose figures that the parallel implementation does not follow this trend. The fastest setup in figure 9 uses row-wise accesses to the left matrix and column-wise accesses to the right matrix. Using the left matrix transpose was only slightly slower, despite changing how the left matrix is accessed.

These results came out the same even when the allocation and prefetching order was swapped for left and right, as well as which elements were grabbed first for multiplication. While shared memory conflicts could cause an issue like this, we used global memory for these tests, so it cannot be the culprit. Rather, we believe it is caused by poor memory coalescing. Memory coalescing is an automatic feature where a group of threads accessing adjacent memory have all their requests grouped into a single transaction. This maximizes memory bandwidth. When accessing non-contiguous memory, this process does not occur and access is serialized, harming performance greatly [4].

Looking closer at how our algorithm actually works, it's clear that in the for loop, the threads grab the ith element of each row and the ith element of each column at once for the dot product. This corresponds to a single column of the left matrix and a single row of the right matrix both being accessed. Grabbing that right row can therefore be coalesced. So, now we must consider a trade-off in our design. Accessing rows grants faster access to global memory for the purposes of loading into shared memory and doing non-shared memory computation. However, based on the memory layout, iterating through a row provides better cache performance. At the moment, it appears that prioritizing coalesced access is better.

9 CONCLUSION

In this work, we designed and evaluated a hybrid MPI and CUDA based matrix multiplication algorithm for large scale computation. Our approach successfully combines distributed memory parallelism with GPU acceleration, enabling significant performance improvements over sequential implementations.

Our results demonstrate that performance is primarily limited by communication overhead, with shared memory providing the most consistent improvement and transposition offering limited benefit.

Overall, our findings indicate that although GPU acceleration greatly enhances computational efficiency, system performance is ultimately constrained by MPI communication costs and memory access behavior. Future improvements should focus on reducing this communication overhead, improving load balancing and improving scalability.

10 FUTURE WORK

All aspects of future development are related to improving the performance of this algorithm. While it works quite well, it could be better. One of the main bottlenecks at this point is communication overhead. MPI is convenient for allowing multi-node computation, but falls short with regards to scaling. Many modern parallel programs opt out of using MPI and design their own message passing integrated into the hardware. While we don't have the funds to build a custom supercomputer to accomplish this, it shows that there are ways to improve communication speed and we shouldn't just accept that MPI is lacking. That being said, the extent of MPI's poor scaling could be the result of us using trivially small matrices.

Future work would also include scaling up our tests to be more intensive and get closer to peak performance, allowing better insight. Related to MPI, we also need to investigate better load balancing to ensure that adding more ranks/GPUs is actually beneficial for the problem at hand.

Next, we must investigate memory issues more closely. As mentioned in the discussion, there appears to be a tradeoff between memory coalescing and optimizing cache hits. It should be possible to achieve both at the same time by reworking thread block geometry. At the moment, we have a one-dimensional thread block that covers the whole tile. A two-dimensional block would do a better job with this by breaking the threads into rows that work together one a single value from the left matrix with a row from the right matrix. In theory, this would result in better memory coalescing. It may also be possible that using less threads in a block would be ideal. Overall, it's necessary that we do further testing with new kernel setups in order to find the best performing one.

One additional issue that we didn't address yet is shared memory bank conflicts. CUDA's shared memory is divided into 32 banks, which the 32 threads in a warp access as needed. A shared memory bank conflict is when two threads in a warp try to access the same memory bank, and their access becomes serialized. We did not consider this when designing the algorithm, so it's very possible that this is harming performance. As noted in the paper by Khan et al., on parallel transpose, bank conflicts arose when tile size was a multiple of 16 [5]. Our tile size is 32, so this applies to our setup. Potential solutions involve adding padding to shared memory or optimizing the order in which threads access it.

TEAM CONTRIBUTIONS

Ryan Koester

Like my teammates, I contributed to the overall planning, code implementation, and experimental evaluation of the project. In particular, I developed the MPI component of the system with feedback from my teammates, including writing the mpi.c module and associated header file, enabling distributed data

partitioning and communication. Also, I assisted in resolving issues with and improving our CUDA kernel implementation to enhance performance. Additionally, I conducted some initial testing to validate performance behavior and test the limits of our system. I helped gather references and write many sections of the report as well.

Ryan Lima
I performed a lot of initial research that contributed to the design of the algorithms. I was also responsible for making the prototypes of the kernels that my teammates improved further. This involved establishing an indexing system that uniquely mapped blocks and their threads to tiles, and determining the ideal number of blocks and threads that should be used for transposing and multiplying. I also parallelized the matrix generation and result checking functions to speed up the program's execution. These aspects are not reflected in the data, but lead to smoother execution for the user. Furthermore, I collected and analyzed the data on caching and indexing to provide insight into why transposing was falling short and where we should look to improve in the future.

Kaden Kwan
For this project, I worked creating the initial outline and formatting for our report such as the sections and bullet point ideas for what we plan to include in each section. I also set up a github repository for our team and worked on the beginning iteration of the MPI code to complement our CUDA code, and wrote instructions on how to run our tests in our README file. Finally, I tested what matrix sizes would work for our test, gathered data for our strong and weak scaling studies, and created the corresponding graphs for each of the test cases we wanted to run.

REFERENCES

[1] N. Tomikj and M. Gusev, "Parallel matrix multiplication," 2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), Opatija, Croatia, 2018, pp. 0204-0209, doi: 10.23919/MIPRO.2018.8400039. Link: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8400039&isnumber=8399814

[2] M. Kim, Y. J. Jang and W. W. Ro, "Parallel transpose of matrix multiplication based on the tiling algorithms," 2011 IEEE 54th International Midwest Symposium on Circuits and Systems (MWSCAS), Seoul, Korea (South), 2011, pp. 1-3, doi: 10.1109/MWSCAS.2011.6026311. Link: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6026311&isnumber=6026262

[3] CCI, RPI . "DCS Cluster - Center for Computational Innovation - Documentation." Rpi.edu, 2024. Link: docs.cci.rpi.edu/clusters/DCS_Supercomputer/

[4] Naznin Fauzia, Louis-Noël Pouchet, and P. Sadayappan. 2015. Characterizing and enhancing global memory data coalescing on GPUs. In Proceedings of the 13th Annual IEEE/ACM International Symposium on Code Generation and Optimization (CGO '15). IEEE Computer Society, USA, 12–22. Link: https://dl.acm.org/doi/pdf/10.5555/2738600.2738603

[5] A. Khan, M. Al-Mouhamed, A. Fatayar, A. Almousa, A. Baqais and M. Assayony, "Padding free bank conflict resolution for CUDA-based matrix transpose algorithm," 15th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), Las Vegas, NV, USA, 2014, pp. 1-6, doi: 10.1109/SNPD.2014.6888709. Link: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&a rnumber=6888709

[6] Austin R. Benson and Grey Ballard, "A framework for practical parallel fast matrix multiplication", 2015 ACM Link: https://dl.acm.org/doi/epdf/10.1145/2858788.2688513

[7] Yinghong Sun and Yuanman Tong, "CUDA based Fast Implementation of Very Large Matrix Computation", 2010, IEEE Link: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=5704475

[8] NVIDIA, "NVIDIA Developer," NVIDIA Developer. https://developer.nvidia.com/ (accessed Apr. 7, 2026)

[9] S. Oral, S. S. Vazhkudai, F. Wang, C. Zimmer, C. Brumgard, J. Hanley, G. Markomanolis, R. Miller, D. Leverman, S. Atchley, and V. V. Larrea, "End-to-end I/O portfolio for the Summit supercomputing ecosystem," in Proceedings of SC19: The International Conference for High Performance Computing, Networking, Storage and Analysis, Denver, CO, USA, 2019, pp. 63:1–63:14, doi: 10.1145/3295500.3356157. Link: https://dl.acm.org/doi/10.1145/3295500.3356157

[10] Open MPI, "Open MPI Documentation," 2024. [Online]. Available: https://www.open-mpi.org/doc/

[11] NVIDIA, "CUDA Toolkit Documentation," 2024. [Online]. Available: https://docs.nvidia.com/cuda/